

DEPARTMENT OF COMPUTER SCIENCE



University of
Reading

CS1IP: Imperative Programming

Week 1: Module Information, Java, Formatted Output, and
Variables

Learning Outcomes

- What is a Computer Programming?
- How to write a program in Java?
- CS1P Module Information

What is Program?

**Imagine yourself telling a Computer to peanut butter
and gem sandwiches?**

Engaging Video: "Dad Computer" executing the "sandwich program"

- https://www.youtube.com/watch?v=cDA3_5982h8&t=5s

A program is sequence of instructions that is executed by a computer

Prior Experience

Some of you will not have written any programs before.

Some of you may already have written hundreds or thousands of lines of program **source code**.

If so, you may find the pace of these lectures too slow, but you can:

1. Make sure you know the correct **terminology**.
2. Learn **good habits** for designing and developing your programs.
3. Familiarise yourself with **Java** if you have only learnt (say) Python.
4. Blitz through the recommended exercises and focus on the optional extension exercises.

Programmers, Coders and Code

Are programmers the same as coders? A recent article discusses...

Coders write program **source code**.

Unless you are describing encryption, encoding or error detection/correction techniques, it is unlikely you are writing **codes**.

Correct:

- I wrote 100 lines of Java **code** in today's programming practical.
- Blu-ray discs use Reed-Solomon error correction **codes**.

Incorrect:

- Can you help me with my **codes** for this programming practical?

How do we write a program?

Basic strategy

1. Type in a sequence of instructions.
2. Tell the computer to execute the instructions.
3. If the program didn't work, fix it and try again.

But:

- How do you know what instructions to write?
- How do you know which parts to edit?

More on this later...

Java

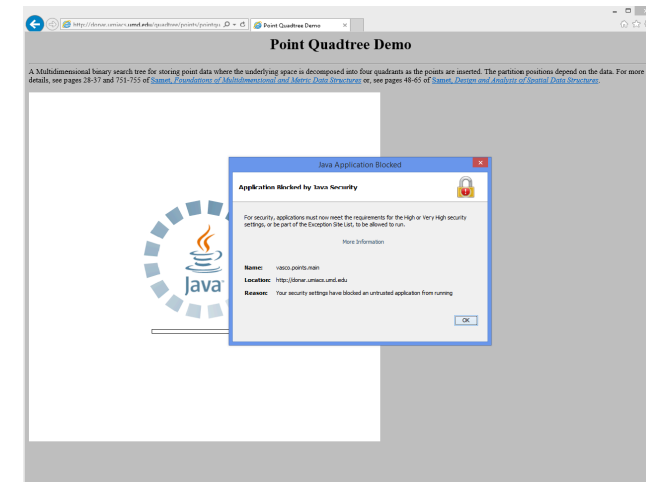
Java was created in 1995 as a language for writing "applets" to add interactivity to early Web pages.

At some point, it was **rebranded** as a **general-purpose programming language**.

- Today Java is popular for **Android apps** and in **large companies**

There are many programming languages, each with its own strengths and weaknesses.

- Today, the dominant language for Web interactivity is JavaScript
- The most popular language for data science and machine



Java

Today Java known for its better support for:

- **static typing**
 - Static typing means the computer can spot more errors at compile time, so you find out about and can fix them faster.
- **object orientation**
 - useful when working on larger projects with a team.

TIOBE Index

Java is high in the TIOBE Index is a ranking of programming language popularity.

Does it matter which programming language you learn?

- Yes — the language you have to express ideas shapes how you think.

Also, some languages provide more support for finding errors early on.

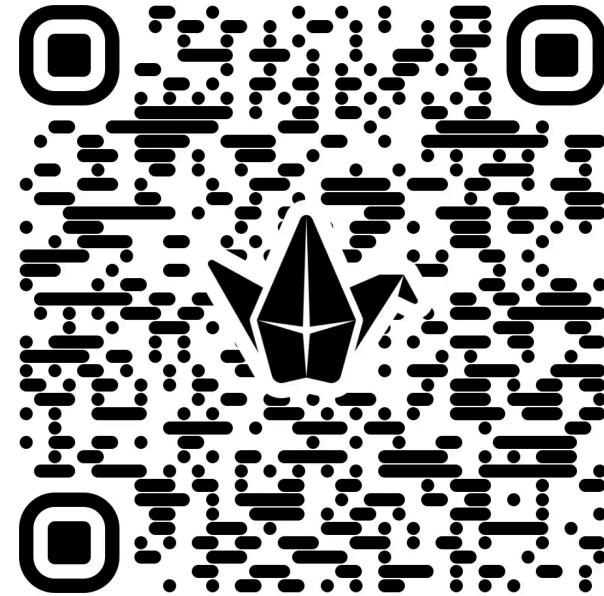
- But once you have learned one imperative language, it is significantly easier to learn a second or third.
- Many popular languages, such as **C**, **C++** and **C#** look like Java. They will be especially easy to learn.

Quiz 1

Which other language are you interested in learn?

- Python
- Rust
- JavaScript
- C++
- C#

 Answer at <https://padlet.com/universityofreading/cs1ip-w1-padlet-u0ptbftlp0lnbeia/slideshow>



Reality check

In practice, you **often don't get to choose which language you use**. If you are working on an existing project or using a particular library, you will usually have to **use the same language as the existing developers**.

- So, learn to program independently of language

Compiled vs Interpreted Languages

Compiled language

- To run a program written in a compiled language, you use another program called a **compiler** to turn your human-readable program **source code** into binary **object code** or machine code that the computer can execute directly.

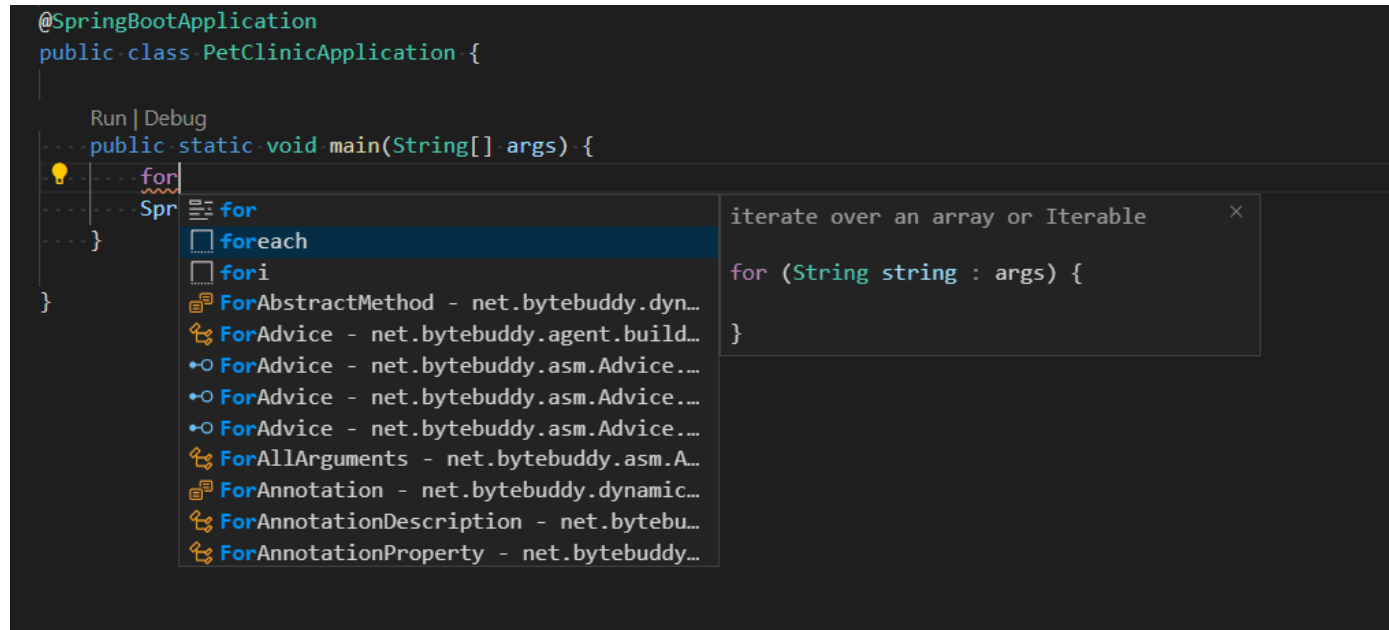
Interpreted language

- To run a program written in an interpreted language, you use another program called an **interpreter**, which executes your source code a line at a time.

compiled languages are faster, but interpreted languages can be more convenient, as there is no separate compilation phase.

How do we write a Java program?

In this module we will use the [Visual Studio Code](#) to write Java programs



The screenshot shows a Visual Studio Code editor with a Java file named `PetClinicApplication.java`. The code is as follows:

```
@SpringBootApplication
public class PetClinicApplication {

    Run | Debug
    public static void main(String[] args) {
        for
    }
}
```

An IDE completion menu is open for the `for` keyword. The menu lists several options:

- `foreach` (highlighted)
- `fori`
- `ForAbstractMethod` - net.bytebuddy.dyn...
- `ForAdvice` - net.bytebuddy.agent.build...
- `ForAdvice` - net.bytebuddy.asm.Advice...
- `ForAdvice` - net.bytebuddy.asm.Advice...
- `ForAdvice` - net.bytebuddy.asm.Advice...
- `ForAllArguments` - net.bytebuddy.asm.A...
- `ForAnnotation` - net.bytebuddy.dynamic...
- `ForAnnotationDescription` - net.bytebu...
- `ForAnnotationProperty` - net.bytebuddy...

A tooltip for the `foreach` option is visible, showing the text: "iterate over an array or Iterable" and a code snippet: `for (String string : args) {`.

We will see more on the Practical.

Java Statements

Sequencing

"A program is a **sequence** of instructions..."

A **straight-line** program is a sequence of instructions executed in a "straight line" from top to bottom.

Later on, we will see programs where the order of execution isn't just the order of lines in the program.

In Java, we call an instruction a **statement** and it always end if a `;`.

Sequencing

This Java statement will print the text "Hello, World!".

```
System.out.println("Hello, World!");
```

This two Java statement will print the text "Hello, World!" and then print "Hello, Students!".

```
System.out.println("Hello, World!");  
System.out.println("Hello, Students!");
```

Breaking down a println() statement

```
System.out.println("Hello, World!");
```

- `System.out.println` is a **function** or **method**.
- `"Hello, World!"` is an **argument** — value passed as input to the function.
 - `"` : Double quotation marks or **quotes** mark starts and end of a text value.
 - The quotes aren't part of the text.
- `()` : Round brackets or parentheses enclose the argument.
 - If I write `System.out.println()` , you immediately know it is a function.
- `;` : A semi-colon **terminates** the statement.

Let's run java a program

To run a Java program, it is required an `public class` with `main` function.

In Java, statements are only allowed inside **methods**, which are only allowed in **classes**. So, thi is a simple Java Program that print the text "Hello, World!"

```
public class HelloWorld {  
    public static void main(String[] args) {  
        System.out.println("Hello, World!");  
    }  
}
```

Every class in Java should be in a file of the same name. So to run, it you should do:

```
java HelloWorld.java
```

Breaking down an `main` function.

```
public class HelloWorld { // class definition
    public static void main(String []args) { // main function
        ...
    } // end of main function
} // end of class
```

`class HelloWorld` with definition `{...}`

- `public` : is available outside the file

`main` function with definition `{...}`

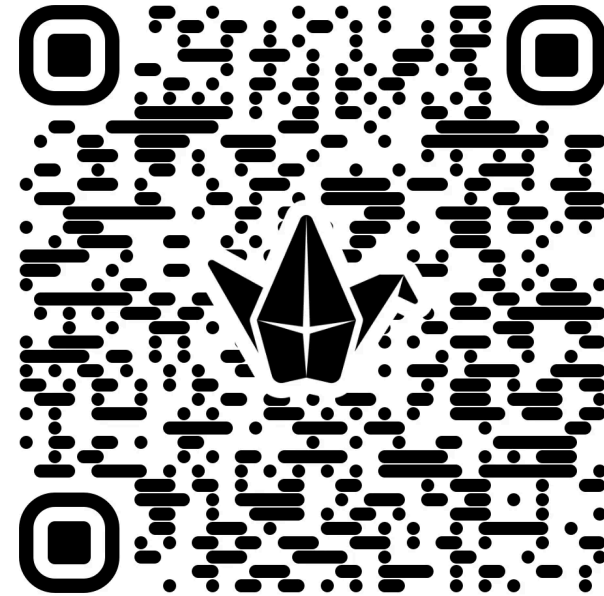
- `public` : is available outside the file
- `static` : a non-class function, which do not need a object
- `void main(String []args)` : special function signature, which will be run when doing `java <FILE>.java`

Quiz 2

Which of this is not program?

- A operational system
- A system Web Interface
- Java
- cmd.exe
- winver

 Answer at <https://padlet.com/universityofreading/cs1ip-w1-padlet-u0ptbftlp0lnbeia/slideshow>



Variables

Variables

A **variable** is a named area for storage in the computer's memory.

Think of it as being like a labelled storage box.

We can put things in the box. If a function "asks" for an argument, we can "tell" it to look in the box.

```
String x;  
x = "Hello."  
System.out.println(x);
```

Here, `x` is the **name** of a variable. `String` is its **type** and tells the computer that `x` is going to be used to hold a text value.

Note we don't put quotes around `x`.

We want to pass the variable `x`, not the text value `"x"`.

Multiple variables and multiple uses

```
String x;  
x = "Hello.";  
System.out.println(x);  
String y;  
y = "Goodbye.";  
System.out.println(y);
```

We can have as many variables as we like.

We can use them as many times as we like.

Reassigning to variables

```
String x;  
x = "Hello.";   
System.out.println(x);  
x = "Goodbye.";   
System.out.println(x);
```

We can change the value of a variable using an **assignment**.

The computer discards the old value.

Remember, straight-line programs are executed from top to bottom. The value of `x` that gets passed to `println()` is the one most recently assigned.

Variable Declaration

In Java, we need to **declare** variables before we can use them.

This causes an error:

```
x = "Hello.";
System.out.println(x);
```

because we forgot the **declaration** `String x;` .

Error messages are your friend!

Don't ignore error messages.

Learn how to understand them and fix your mistakes.

Declaration, Initialisation and Assignment

Declaring a variable requires a **name** and **type**:

```
String x;
```

Assigning to a variable sets its **value**:

```
x = "Hello.";
```

Initialising a variable simultaneously declares and assigns:

```
String x = "Hello.";
```

Breaking down variable initialisation

```
String x = "Hello.";
```

- `String` is the **type** of the variable.
 - We can only assign values of type `String` to this variable.
- `x` is the **name** of the variable.
 - We can choose any *reasonable* name, not just single letters.
 - Use meaningful names to make your program easier to understand.
- `=` indicates **assignment**, which is part of initialisation.
- `"Hello."` is the initial value of the variable.
- `;` terminates the statement.

Types

So far, we have only seen the type `String`:

- `String` — a text value; a sequence of characters

Here are some more:

- `int` — an **integer** value (whole number)
- `boolean` — either `true` or `false`

Mathematically, you can think of a **type** as the set of all possible values a variable is allowed to hold.

Using different types

We can declare variables of different types and print their values:

```
String university = "Reading";  
int year = 1892;  
System.out.println("The University of ");  
System.out.println(university);  
System.out.println("was founded in ");  
System.out.println(year);  
System.out.println(".");
```

`println()` vs `print()`

`println()` prints a special **newline** character (which starts a new line on the screen) after printing its argument. `print()` does not.

```
String university = "Reading";  
int year = 1892;  
System.out.print("The University of ");  
System.out.print(university);  
System.out.print("was founded in ");  
System.out.print(year);  
System.out.println(".");
```

Operators, Expressions, Assignment

Operators and Expressions

So far, we have assigned values to variables and printed them out.

Usually, we want to do some kind of calculation or computation with values.

We can use the normal mathematical **operators** (+, -, *, /) with constants and variables to build **expressions**.

We can also use + to **concatenate** (join together) strings.

```
int weeks = 12;  
int days = weeks * 7;  
System.out.println("In " + weeks + " weeks there are " + days + " days.");
```

Java will convert values to strings if necessary for concatenation.

Precedence

Expressions follow the usual mathematical rules for **precedence** ("BODMAS"), so this works as you would expect:

```
int a = 48;  
int b = 40;  
int aab = 2 * a + 1 * b;  
System.out.println("AAB is " + aab + " UCAS points.");
```

If you're not sure, just use brackets:

```
int aab = (2*a) + (1*b);
```

But consider this:

```
System.out.println("AAB is " + 2*a + 1*b + " UCAS points.");
```

Division

```
int today = 275;  
int xmas = 360;  
int weeks = (xmas-today)/7;  
System.out.println("There are " + weeks + " weeks until Christmas.");
```

Integer division rounds towards zero.

Use the modulus operator (`%`) to get the remainder from division.

```
int today = 275;  
int xmas = 360;  
int weeks = (xmas-today)/7;  
int days = (xmas-today)%7;  
System.out.print("There are " + weeks + " weeks ");  
System.out.print("and " + days + " day(s) until Christmas.");
```

Floating Point

When you **need** non-integer numbers, use `double` for **double-precision floating point** variables.

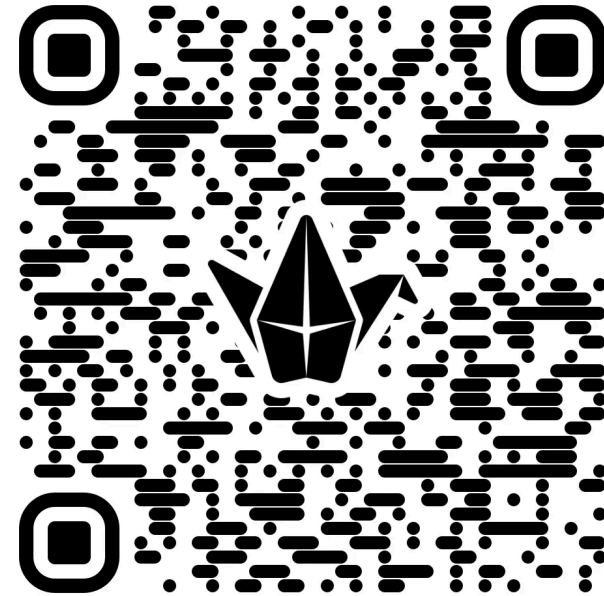
```
double inches = 12.0;
double cm_per_inch = 2.54;
double cm = inches * cm_per_inch;
System.out.println(inches + " inches is " + cm + " centimetres.");
```

Quiz 3

Which is the value of x in this code: `double
x = 1/10; ?`

- 0.1
- 0
- 0.01

i Answer at https://padlet.com/universityofreading/cs1ip-w1-padlet-u0ptbftlp0lnbeia/slideshow_qr_code.png



Floating Point Pitfalls

But be careful! Every floating point calculation is rounded.

- You may need to write `.0` after an integer constant to force Java to treat it as a floating-point value.

```
double x = 1/10;  
System.out.println(x);
```

Every calculation is rounded.

```
double t = 1.0/10.0;  
double one = t+t+t+t+t+t+t+t+t;  
double zero = 1 - one;  
System.out.println(zero);
```

Updating a variable

What does this do?

```
int x = 1;  
x = x + 1;  
System.out.println(x);
```

Mathematically, $x = x + 1$ is nonsense. But while we may read it as:

x equals x plus one

what it means is:

evaluate $x + 1$ and assign the result to x .

Formatted Output

Formatted Output — `printf()`

Using `+` to join together lots of tiny strings is tedious and error-prone.

Formatted output using `printf()` makes this much easier. Compare:

```
int weeks = 12;  
int days = weeks * 7;  
System.out.println("In " + weeks + " weeks there are " + days + " days.");
```

with:

```
int weeks = 12;  
int days = weeks * 7;  
System.out.printf("In %d weeks there are %d days.%n", weeks, days);
```

Breaking down a printf() statement

```
System.out.printf("In %d weeks there are %d days.%n", weeks, days);
```

- `System.out.printf` is a **function** or **method**.
- `()` : Round brackets or parentheses enclose the **arguments**.
 - When a function has multiple arguments, separate with a comma (`,`).
- `;` : A semi-colon **terminates** the statement.
- `"In %d weeks there are %d days.%n"` is a **format string**.
 - `%d` is a **format specifier**. It means "substitute the next argument, which is a (decimal) integer, in here".
 - `%n` is also a format specifier. It means "substitute a newline here".
- The remaining arguments (`weeks` and `days`) get turned into strings and substituted into the format string.

More format specifiers

Use `%s` for strings and `%f` for floating point.

```
String uni = "Reading";
int year = 1892;
System.out.printf("The University of %s was founded in %d.%n", uni, year);

double loan = 13348;
double interest = 7.3;
double compound = 1 + (interest/100);
double owed = loan * compound;
owed = (owed + loan) * compound;
owed = (owed + loan) * compound;
System.out.printf("In 3 years, your loan will be %f pounds.%n", owed);
```

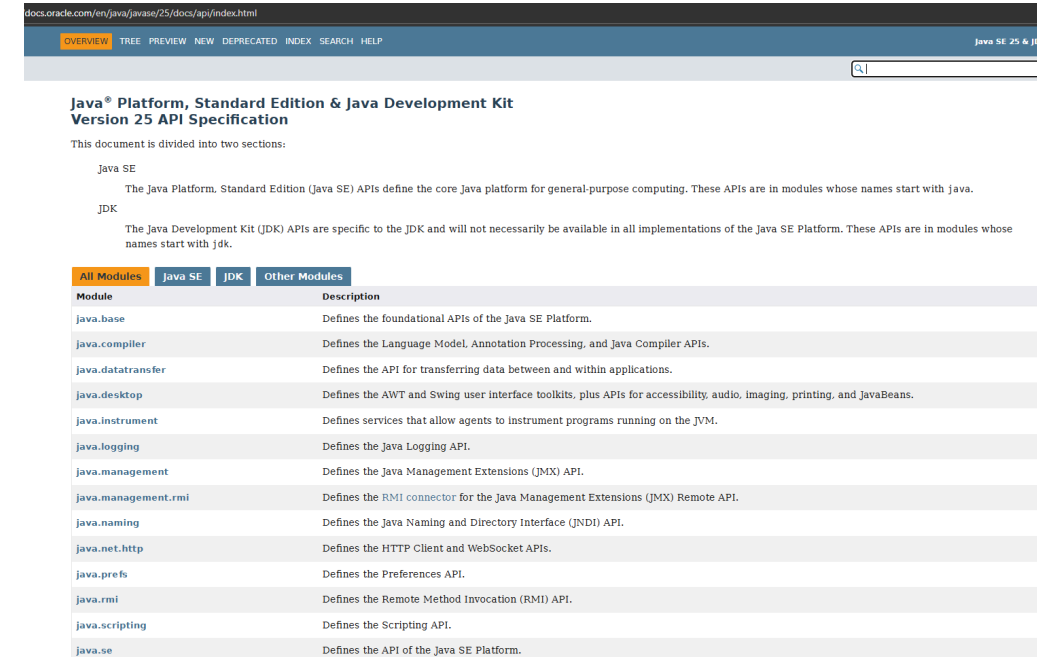
There are many more. [Consult the documentation!](#)

Documentation, Errors, Debugging

Java Documentation and other sources of information

1. look at the recommended [Textbook](#)
2. look at official [Java documentation](#)
3. ask a friend;
4. search online for tutorials and programming forums;
5. ask an AI chatbot, such as ChatGPT.

Sources near the top of the list are more likely to be reliable and relevant.



The screenshot shows the Oracle Java SE 25 API Specification page. The page has a dark header with navigation links: OVERVIEW, TREE, PREVIEW, NEW, DEPRECATED, INDEX, SEARCH, and HELP. The main title is "Java® Platform, Standard Edition & Java Development Kit Version 25 API Specification". Below the title, it states "This document is divided into two sections:" and lists "Java SE" and "JDK". The "JDK" section is expanded, showing a table of modules. The table has two columns: "Module" and "Description". The modules listed are: java.base, java.compiler, java.datatransfer, java.desktop, java.instrument, java.logging, java.management, java.management.rmi, java.naming, java.net.http, java.prefs, java.rmi, java.scripting, and java.se.

Module	Description
java.base	Defines the foundational APIs of the Java SE Platform.
java.compiler	Defines the Language Model, Annotation Processing, and Java Compiler APIs.
java.datatransfer	Defines the API for transferring data between and within applications.
java.desktop	Defines the AWT and Swing user interface toolkits, plus APIs for accessibility, audio, imaging, printing, and JavaBeans.
java.instrument	Defines services that allow agents to instrument programs running on the JVM.
java.logging	Defines the Java Logging API.
java.management	Defines the Java Management Extensions (JMX) API.
java.management.rmi	Defines the RMI connector for the Java Management Extensions (JMX) Remote API.
java.naming	Defines the Java Naming and Directory Interface (JNDI) API.
java.net.http	Defines the HTTP Client and WebSocket APIs.
java.prefs	Defines the Preferences API.
java.rmi	Defines the Remote Method Invocation (RMI) API.
java.scripting	Defines the Scripting API.
java.se	Defines the API of the Java SE Platform.

Errors

Compile-Time Errors

When you try to run a program, the computer checks for obvious **compile-time** errors. Errors come with a message describing the problem. Read it and understand it.

```
System.out.println("Speed of sound (m/s):  + 330);
```

Run-Time Errors

Sometimes the computer can't spot errors until it starts running the program. These errors are called **run-time** errors.

```
System.out.println("Speed of sound (m/s): " + 330);  
System.out.println("Speed of light (m/s): " + 300000000);  
System.out.println("To infinity and beyond:" + 1/0);
```

Errors

Semantic Errors

Sometimes your program will run successfully, but not do what you expected. These errors are hardest to fix.

The computer does not know what you really meant. It will do exactly what you told it to do.

Large applications may have 100,000 lines of code or more.

Making mistakes is an inherent part of programming. Test your programs often while you are writing them to find mistakes as quickly as possible.



CS1IP Module Information

Imperative Programming

What is **imperative programming**?

- It is a **sequence of instructions** that is executed by a computer.
- Most mainstream programming languages are imperative.
- Java is an **imperative** and **object-oriented** language.
- This module will focus on the **imperative** part of Java.

What is **object-oriented programming**?

- uses **Objects** and **classes**, which help to split large programs up into small, more manageable, easily reusable and extensible parts.
- You will learn about objects in **CS1OP: Object-oriented Programming**.

What not is Imperative Programming

A **declarative program** is a sequence of declarations that describe a goal. The computer works out what steps to use to get to the goal.

- Example: **SQL** (Structured Query Language) — for querying and updating databases.

- ```
SELECT email FROM employees WHERE department = 'Sales' AND hire_date > '2020-01-01' ORDER BY last_name ASC;
```

# CS1IP: Imperative Programming

## Timetabled teaching

- 11 x 2-hour lectures — Weeks 1-5, 7-12
- 10 x 2-hour practicals — Weeks 1-5, 7-12
- No undergraduate teaching in Week 6

## Assessment

- Programming exercise — 30% — Published on at Week 2 and Hand-in Week 7
- Programming exercise — 40% — released on BB at Week 7 and Hand-in Week 11
- In-person multiple choice exam — 30% — Assessment Period

## CS1IP: Help available

- Practicals are your main opportunity to interact with lecturers and other teaching staff. Don't waste your tuition fees by not showing up.
- You can ask the lecturer before/after lectures if there is time.
- Lecturer's office hours — drop in or make appointment by e-mail.
- E-mail the lecturer — simple questions only.
- Make friends and discuss the module with them.

# CS1IP: Resources

## Recommended Textbook

- [Textbook "Java How To Program, Late Objects, Global Edition, 11th Edition, by Deitel"](#)

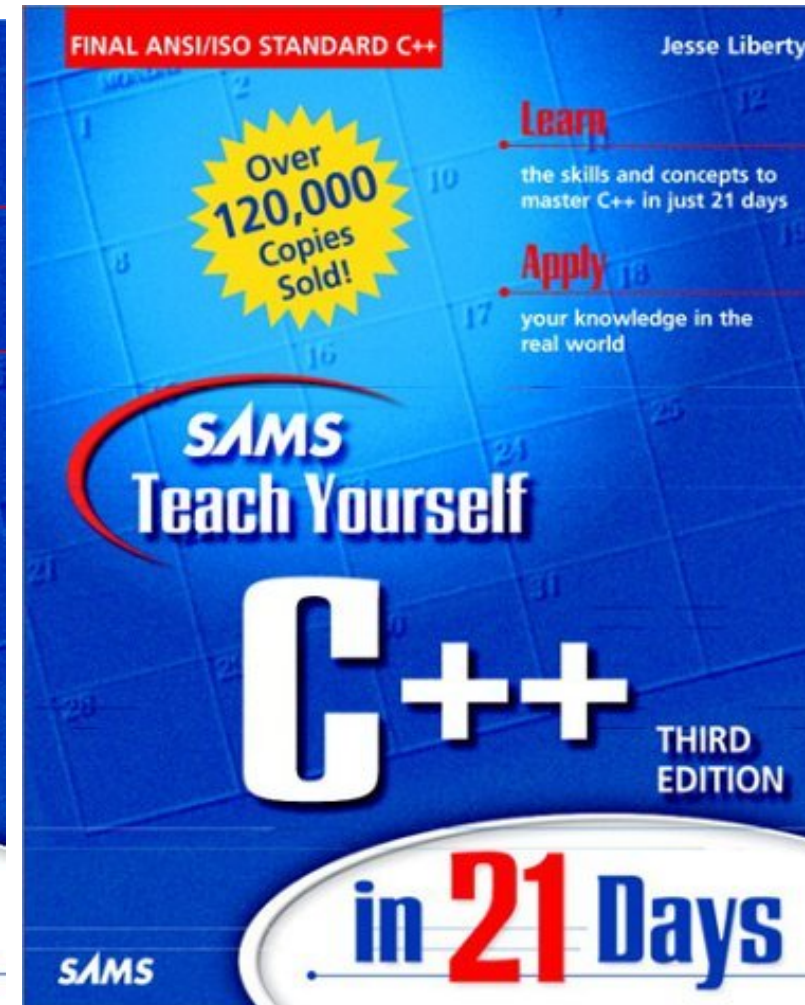
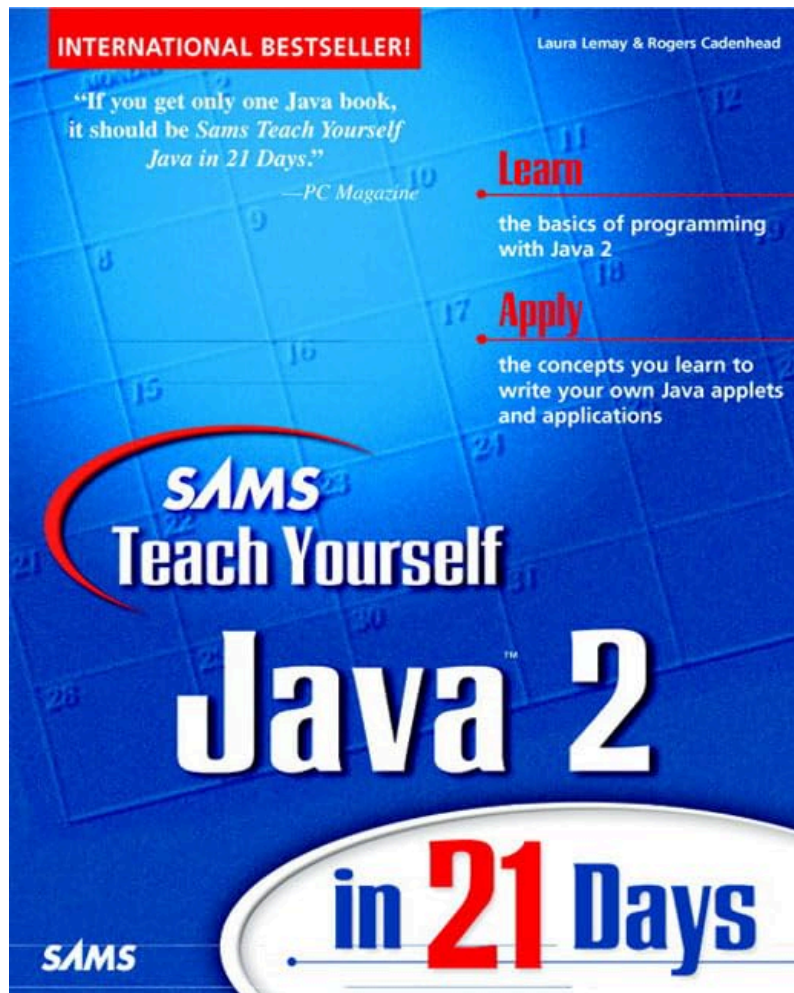
## Blackboard

- Lecture slides
- Practical exercises
- Recommended reading
- Self-study exercises

# How to do well

- Attend every lecture and pay attention.
  - If you miss a lecture, watch the recording the same week.
- Review the lecture slides and quizzes after the lecture.
- Attend every practical and attempt the recommend exercises.
  - If you get stuck, spend a few minutes trying and then ask friends or demonstrators.
  - If necessary, complete the exercises independently after the practical.
- Read the recommended [Textbook](#) chapter/excerpt every week.

# Learn Programming in 21 Days?



# Learn programming in 11 weeks?

We can teach you the main features of the Java programming language.

We can't teach you to become good at programming.

That takes years of independent practice on projects that interest you.

Your journey starts now.

# Review Learning Outcomes

# Review Learning Outcomes

| What is a Program?

**A sequence of instructions that is executed by a computer**

| How to program in Java?

**Using Statements inside the main function of a public class**

| What is CS1P?

**CS1P focus on the imperative part of Java, while the CS1OOP at the 2nd semester will focus on Object-Oriented part of Java**

**DEPARTMENT OF COMPUTER SCIENCE**



Thank you! 😊

Do you have any questions 🙋 ?

*The next lecture will cover: Code selection, editor and git*